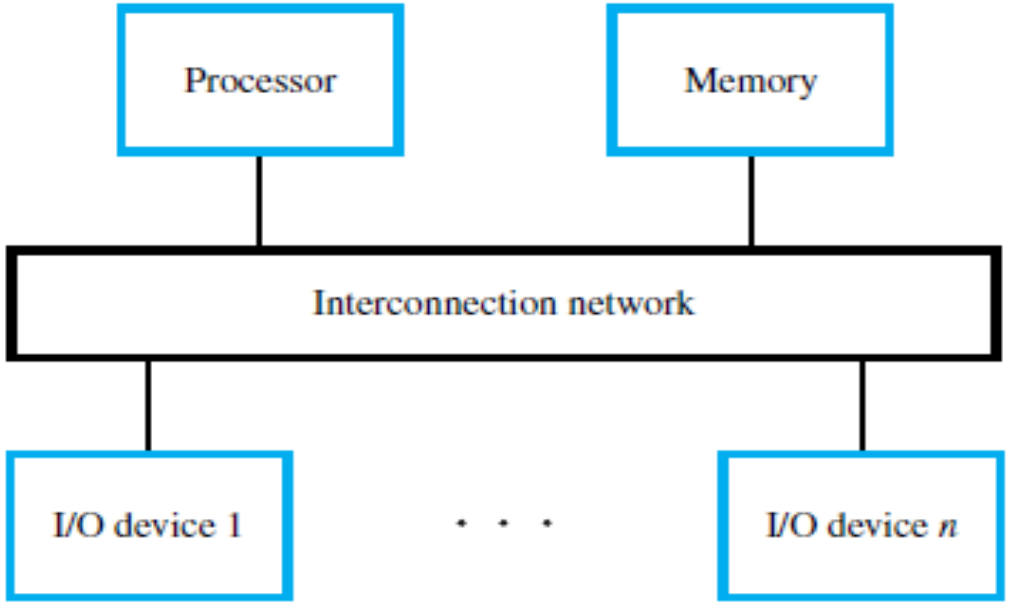
MOD4

Input / Output - External Devices; I/O Modules; Programmed I/O, Interrupt Driven I/O; Direct Memory Access;

Embedded I/O Systems -Embedded I/O, General Purpose I/O, Serial I/O, Other Peripherals.

INTRODUCTION

- The components of a computer system communicate with each other through an interconnection network
- The interconnection network consists of circuits needed to transfer information between the processor, the memory unit, and a number of I/O devices



Input / Output (I/O)

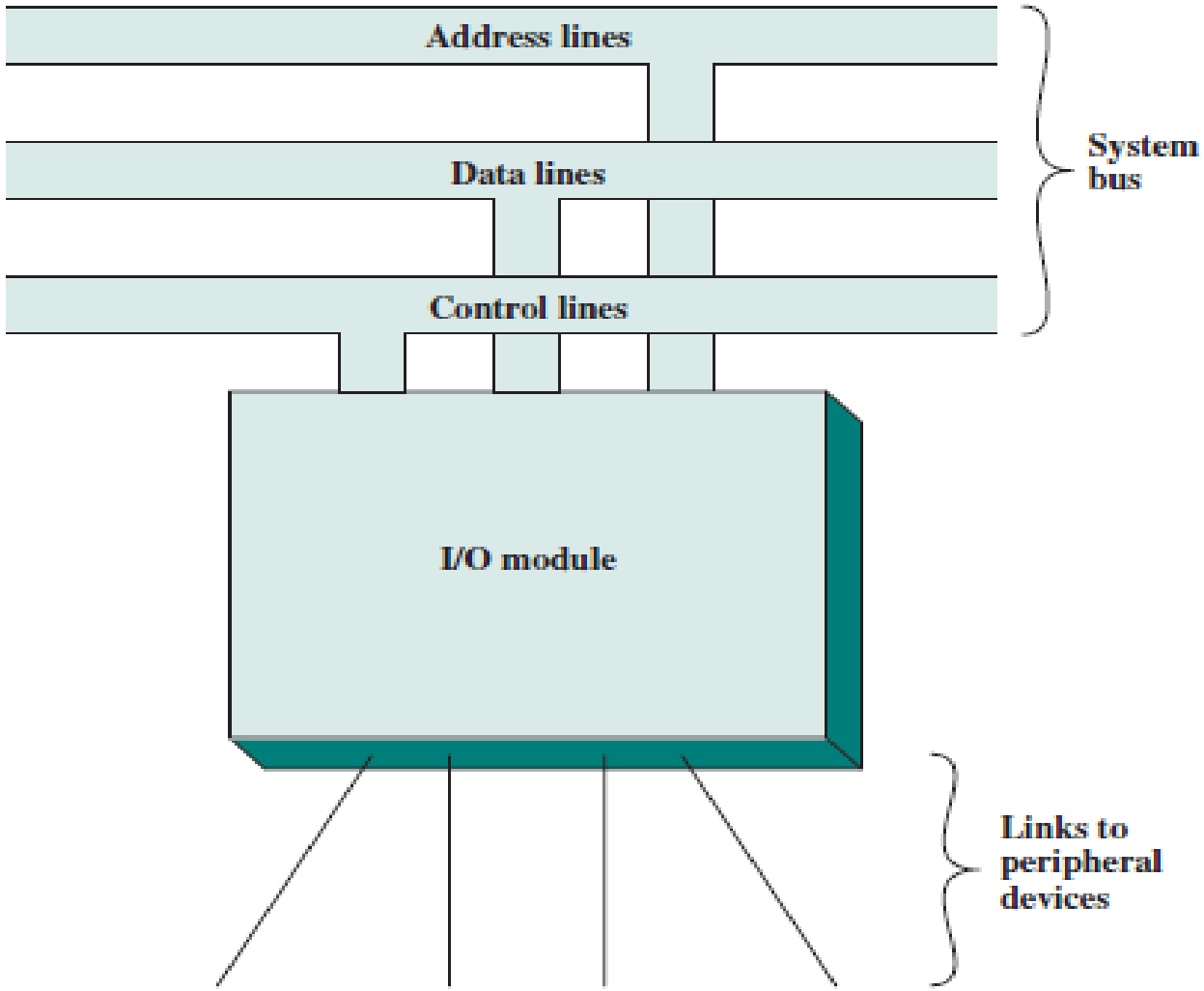
- **Input / Output (I/O)** refers to the communication between a **computer system and external devices**.
- **Input** → Data entering the computer
- **Output** → Data leaving the computer
- The I/O system enables the **CPU, memory, and external devices** to exchange data.

<u>Components of I/O System</u>

1. External Devices
2. I/O Module (Controller)
3. System Bus

1. External Devices

- I/O operations are accomplished through a wide assortment of external devices that provide a means of exchanging data between the external environment and the computer.
- External devices are peripheral devices connected to a computer system used for input, output, storage, or communication.
- An external device attaches to the computer by a link to an I/O module.
- The link is used to exchange control, status, and data between the I/O module and the external device.
- An external device connected to an I/O module is often referred to as a *peripheral device* or, simply, a *peripheral.*
- Examples: Keyboard Mouse Printer Hard disk Network card


Address lines
Data lines
Control lines
System bus
I/O module
Links to peripheral devices

2. System Bus

- The **system bus** connects the CPU, memory, and I/O modules.
- The **bus** carries data and control signals between: CPU, Memory. I/O modules
- It consists of three types of lines:

1. Address Lines

- Used to **identify the specific I/O device or memory location**.
- The CPU sends the **address of the device** through these lines.

2. Data Lines

- Used to **transfer actual data** between: CPU, MEMORY, I/O DEVICES
- Data transfer can occur **in both directions**.

3. Control Lines

- Used to **control and coordinate operations**.

  Examples of control signals: Read, Write, Interrupt, Acknowledge

3. I/O Module

- The **I/O module** acts as an **interface between the system bus and peripheral devices**.
- Each module interfaces to the system bus or central switch and controls one or more peripheral devices.
- An I/O module is not simply a set of mechanical connectors that wire a device into the system bus. Rather, the I/O module contains logic for performing a communication function between the peripheral and the bus
-

  Functions:

➢ Control and timing

➢ Data transfer

➢ Device communication

➢ Buffering data

➢ Error detection

Classification of External Devices

- External devices are classified into three main types:

1. **Human-readable devices**
2. **Machine-readable devices**
3. **Communication devices**

1. Human-Readable Devices

- Devices used for interaction with users and the computer.
- Examples:
- Keyboard
- Monitor
- Printer

2. Machine-readable devices

- Used for communication between the computer and equipment.
- Devices used by other machines.
- Examples: Magnetic disk and tape systems., Sensors and actuators.

3. Communication Devices

- Devices used for data communication.
- Communication devices allow a computer to exchange data with a remote device, which may be a human-readable device, such as a terminal, a machine readable device, or even another computer.
- Examples:

➢ Modem

➢ Network Interface Card

➢ Wi-Fi adapter

External Device

- The interface to the I/O module and external device is in the form of **control, data, and status signals**.
- *Control signals* determine the function that the device will perform, such as send data to the I/O module (INPUT or READ), accept data from the I/O module (OUTPUT or WRITE), report status, or perform some control function particular to the device (e.g., position a disk head).
- *Data* are in the form of a set of bits to be sent to or received from the I/O module.
- *Status signals* indicate the state of the device.
- Examples are READY/NOT-READY to show whether the device is ready for data transfer.

- *Control logic* associated with the device controls the device's operation in response to direction from the I/O module.
- The *transducer* converts data from electrical to other forms of energy during output and from other forms to electrical during input.
- Typically, a buffer is associated with the transducer to temporarily hold data being transferred between the I/O module and the external environment.

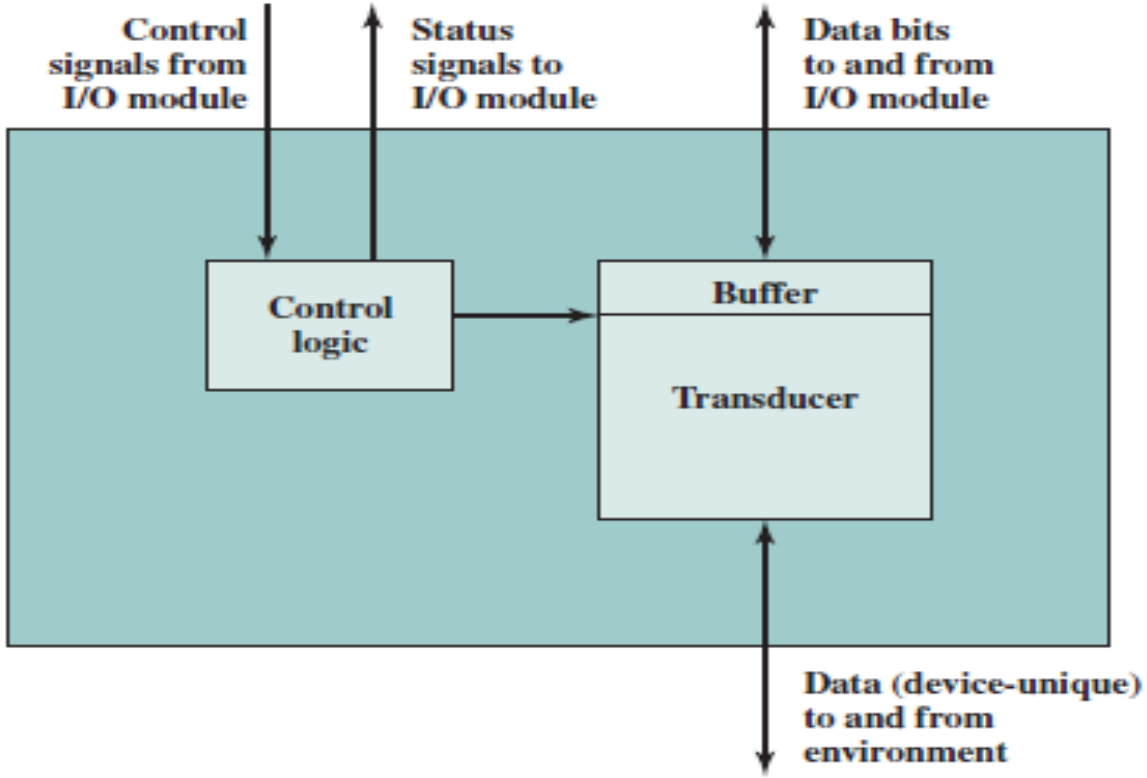


Figure 7.2 Block Diagram of an External Device

Keyboard/Monitor

- The most common means of computer/user interaction is a keyboard/monitor arrangement. The user provides input through the keyboard.
- This input is then transmitted to the computer and may also be displayed on the monitor.
- In addition, the monitor displays data provided by the computer.
- The basic unit of exchange is the character.
- Associated with each character is a code, typically **7 or 8 bits in length**.
- The most commonly used text code is the **International Reference Alphabet (IRA**) (international slandered version of ASCII)
- 1 Each character in this code is represented by a unique 7-bit binary code; thus, (2^7) 128 different characters can be represented.

Keyboard/Monitor

Characters are of two types:

- Printable characters and control characters

1. Printable characters

- the alphabetic, numeric, and special characters that can be printed on paper or displayed on a screen.

2. control characters

- have to do with controlling the printing or displaying of characters or communication ;
- an example is carriage return.

Keyboard/Monitor interaction

For keyboard input,

1. when the user depresses a key, this generates an electronic signal.
2. The signal is interpreted by the transducer in the keyboard and translated into the bit pattern of the corresponding IRA code.
3. This bit pattern is then transmitted to the I/O module in the computer.
4. At the computer, the text can be stored in the same IRA code.

On output,

1. IRA code characters are transmitted to an external device from the I/O module.
2. The transducer at the device interprets this code
3. sends the required electronic signals to the output device either to display the indicated character or perform the requested control function.

Disk Drive

- A disk drive contains electronics for exchanging data, control, and status signals with an I/O module plus the electronics for controlling the disk read/write mechanism.
- In a fixed-head disk, the transducer is capable of converting between the magnetic patterns on the moving disk surface and bits in the device's buffer .
- A moving-head disk must also be able to cause the disk arm to move radially in and out across the disk's surface

I/O MODULES

- An **I/O module** is the hardware component that **interfaces external devices with the CPU and main memory**.
- The major functions or requirements for an I/O module fall into the following categories:

1. Control and timing
2. Processor communication
3. Device communication
4. Data buffering
5. Error detection

1.control and timing

- The processor may communicate with one or more external devices in unpredictable patterns, depending on the program's need for I/O.
- The internal resources, such as main memory and the system bus, must be shared among a number of activities, including data I/O.
- Thus, **control and timing** requirement, to coordinate the flow of traffic between internal resources and external devices.
- If the system employs a bus, then each of the interactions between the processor and the I/O module involves one or more bus arbitrations.
- The I/O module must communicate with the processor and with the external device.

Steps of data transfer data from an external device to the processor

1. The processor interrogates the I/O module to check the status of the attached device.

2. The I/O module returns the device status.

3. If the device is operational and ready to transmit, the processor requests the transfer of data, by means of a command to the I/O module.

4. The I/O module obtains a unit of data (e.g., 8 or 16 bits) from the external device.

5. The data are transferred from the I/O module to the processor.

2. Processor communication

❑The I/O module communicates with the processor in the following

➢**Command decoding:** The I/O module accepts commands from the processor, typically sent as signals on the control bus.

❖For example

- an I/O module for a disk drive might accept the following commands:

✓READ SECTOR, WRITE SECTOR, SEEK track number, and SCAN record ID.

✓The latter two commands each include a parameter that is sent on the data bus.

➢ **Data:** Data are exchanged between the processor and the I/O module over the data bus.

➢ Status reporting:

- since peripherals are so slow, it is important to know the status of the I/O module.
  - if an I/O module is asked to send data to the processor (read), it may not be ready to do so because it is still working on the previous I/O command. This fact can be reported with a status signal.
- Common status signals are BUSY and READY.
- There may also be signals to report various error conditions.

➢ Address recognition:

- Just as each word of memory has an address, so does each I/O device.
- Thus, an I/O module must recognize one unique address for each peripheral it controls.

3. Device communication

- The I/O module must be able to perform **device communication**.
- This communication involves commands, status information, and data.

4. Data buffering

- An essential task of an I/O module is **data buffering**.
- Whereas the transfer rate into and out of main memory or the processor is quite high, the rate is orders of magnitude lower for many peripheral devices and covers a wide range.
- Data coming from main memory are sent to an I/O module in a rapid burst.
- The data are buffered in the I/O module and then sent to the peripheral device at its data rate.
- In the opposite direction, data are buffered so as not to tie up the memory in a slow transfer operation.
- Thus, the I/O module must be able to operate at both device and memory speeds.
- Similarly, if the I/O device operates at a rate higher than the memory access rate, then the I/O module performs the needed buffering operation.

5. Error detection

- I/O module is often responsible for **error detection** and for subsequently reporting errors to the processor.
- One class of errors includes mechanical and electrical malfunctions reported by the device (e.g., paper jam, bad disk track).
- Another class consists of unintentional changes to the bit pattern as it is transmitted from device to I/O module.
- Some form of error-detecting code is often used to detect transmission errors.
- A simple example is the use of a parity bit on each character of data.
- For example, the IRA character code occupies 7 bits of a byte.
- The eighth bit is set so that the total number of 1s in the byte is even (even parity) or odd (odd parity).
- When a byte is received, the I/O module checks the parity to determine whether an error has occurred.

I/O Module Structure

- I/O modules vary considerably in complexity and the number of external devices that they control.

<u>General block diagram of an I/O module</u>

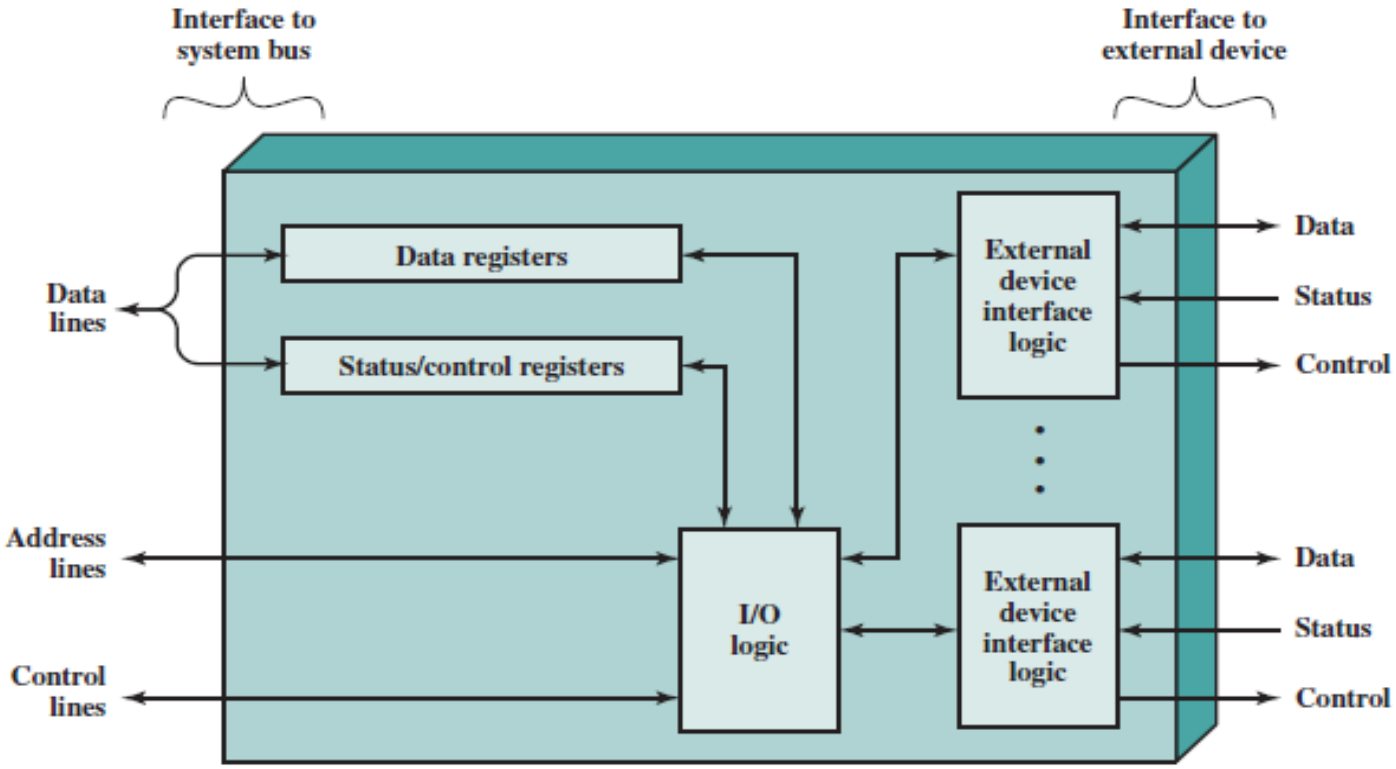


Figure 7.3 Block Diagram of an I/O Module

- The module connects to the rest of the computer through a set of signal lines (e.g., system bus lines).
- **Data transferred to and from the module are buffered** in one or more data registers.
- There may also be one or more status registers that provide current status information.
- A status register may also function as a control register, to accept detailed control information from the processor.
- The logic within the module interacts with the processor via a set of control lines.
- The processor uses the control lines to issue commands to the I/O module.
- Some of the control lines may be used by the I/O module (e.g., for arbitration and status signals).

- The module must also be able to recognize and generate addresses associated with the devices it controls.
- Each I/O module has a unique address or, if it controls more than one external device, a unique set of addresses.
- I/O module contains logic specific to the interface with each device that it controls.
- An I/O module functions to allow the processor to view a wide range of devices in a simple-minded way.
- The I/O module may hide the details of timing, formats, and the electro mechanics of an external device so that the processor can function in terms of simple read and write commands, and possibly open and close file commands.
- In its simplest form, the I/O module may still leave much of the work of controlling a device visible to the processor.

- *I/O channel* or *I/O processor*
  - An I/O module that takes on most of the detailed processing burden
  - presenting a high-level interface to the processor.

- *I/O controller* or *device controller*
  - An I/O module that is quite primitive and requires detailed control.
  - commonly seen on microcomputers, whereas I/O channels are used on mainframes.

Techniques for I/O operations

Three techniques are possible for I/O operations.

- ❑Programmed I/O
- ❑Interrupt-Driven I/O
- ❑Direct Memory Access

PROGRAMMED I/O

- With *programmed I/O*, data are exchanged between the processor and the I/O module.
- In **Programmed I/O**, data transfer occurs **directly between the processor and the I/O module** under the control of a program.
- The processor executes a program that gives it direct control of the I/O operation, including sensing device status, sending a read or write command, and transferring the data.
- When the processor issues a command to the I/O module, it must wait until the I/O operation is complete.

- **Working**
- CPU sends a **command** to the I/O module.
- CPU **checks the device status repeatedly (polling)**.
- When the device becomes ready, **data is transferred between CPU and I/O module**.
- CPU **waits until the operation completes**.
- **Characteristics**
- CPU is **fully involved** in the transfer.
- CPU performs **busy waiting**.
- **Disadvantage**
- **Waste of CPU time** because the processor waits for slow I/O devices.

INTERRUPT-DRIVEN I/O

- In **Interrupt-Driven I/O**, the CPU **does not continuously check the device**. Instead, the **I/O device sends an interrupt signal to the CPU when it is ready**.
- The processor sends an I/O command to the I/O module.
- The processor continues executing other instructions.
- When the I/O operation is complete, the I/O module interrupts the processor.
- The processor handles data transfer between main memory and the I/O module

➢ **Working**

- CPU sends an **I/O command**.
- CPU **continues executing other programs**.
- When the device is ready, it **sends an interrupt signal**.
- CPU temporarily **stops current execution and runs the Interrupt Service Routine (ISR)**.
- Data transfer occurs between **CPU and I/O module**.

➢ **Advantages**

- **No busy waiting**.
- **Better CPU utilization** compared to Programmed I/O.

➢ **Limitation**

- CPU still **handles the data transfer**, so CPU time is still used.

Direct Memory Access

- In **DMA**, data is transferred **directly between I/O device and main memory** without continuous CPU involvement
- The I/O module and main memory transfer data directly.
- The processor is not involved in the actual data transfer.
- This reduces the processor's workload

➢Working

- CPU initializes the **DMA controller** by giving:
  - Memory address
  - Number of bytes
  - Direction (read/write)
- DMA controller **takes control of the system bus**.
- Data transfers **directly between I/O device and memory**.
- When the transfer is complete, DMA **interrupts the CPU**.

➢Advantages

- **Very fast data transfer**.
- **CPU is free to execute other tasks**.
- Efficient for **large data transfers**.

	No Interrupts	Use of Interrupts
I/O-to-memory transfer through processor	Programmed I/O	Interrupt-driven I/O
Direct I/O-to-memory transfer		Direct memory access (DMA)

Overview of Programmed I/O

- When the processor is executing a program and encounters an instruction relating to I/O, it executes that instruction by issuing a command to the appropriate I/O module.
- With programmed I/O, the I/O module will perform the requested action and then set the appropriate bits in the I/O status register.
- The I/O module takes no further action to alert the processor.
- Thus, it is the responsibility of the processor periodically to check the status of the I/O module until it finds that the operation is complete.

I/O Commands- issued by the processor to the I/O module

- To execute an I/O-related instruction, the processor issues an address, specifying the particular I/O module and external device, and an I/O command.
- There are four types of I/O commands that an I/O module may receive when it is addressed by a processor.

1. **Control:**

➢ Used to activate a peripheral and tell it what to do.

➢ These commands are tailored to the particular type of peripheral device

For example, a magnetic-tape unit may be instructed to rewind or to move forward one record.

2. **Test:**

- Used to test various status conditions associated with an I/O module and its peripherals.
- The processor will want to know that the peripheral is **powered on and available for use.**
- It will also want to know if the **most recent I/O** operation is completed and if any errors occurred.

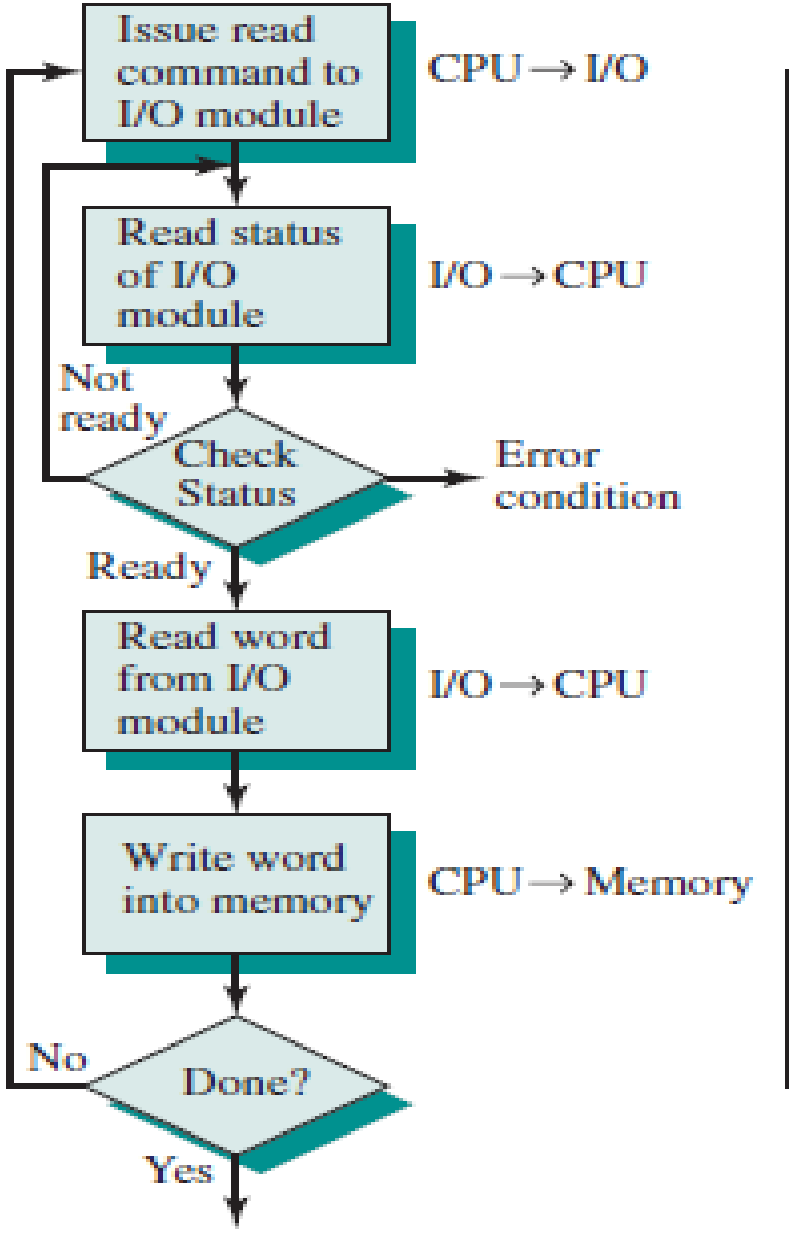
3. **Read:**

- I/O module to obtain an item of data from the peripheral
- place it in an internal buffer
- The processor can then obtain the data item by requesting that the I/O module place it on the data bus.

4. **Write:**

- The I/O module to take an item of data (byte or word) from the data bus
- Subsequently transmit that data item to the peripheral.

- Data are read in one word at a time.
- For each word that is read in, the processor must remain in a status-checking cycle until it determines that the word is available in the I/O module's data register.



(a) Programmed I/O

I/O Instructions- executed by the processor.

- With programmed I/O, there is a close correspondence between the I/o-related Instructions that the processor fetches from memory and the I/O commands that the Processor issues to an I/O module to execute the instructions.
- That is, the instructions are easily mapped into I/O commands, and there is often a simple one-to-one Relationship.
- The form of the instruction depends on the way in which external devices are addressed.
- Each device is given a unique identifier or address.
- When the processor Issues an I/O command, the command contains the address of the desired device.
- Thus, each i/o module must interpret the address lines to determine if the command is for itself.

- When the processor, main memory, and I/O share a common bus, two modes of addressing are possible.

1. **memory mapped**

- With **memory-mapped I/O**, there is a single address space for memory locations and I/O devices.
- The processor treats the status and data registers of I/O modules as memory locations and uses the same machine instructions to access both memory and I/O devices.
- So, for example, with 10 address lines, a combined total of 2^10 = 1024 memory locations.
- and I/O addresses can be supported, in any combination.
- With memory-mapped I/O, a single read line and a single write line are needed on the bus.

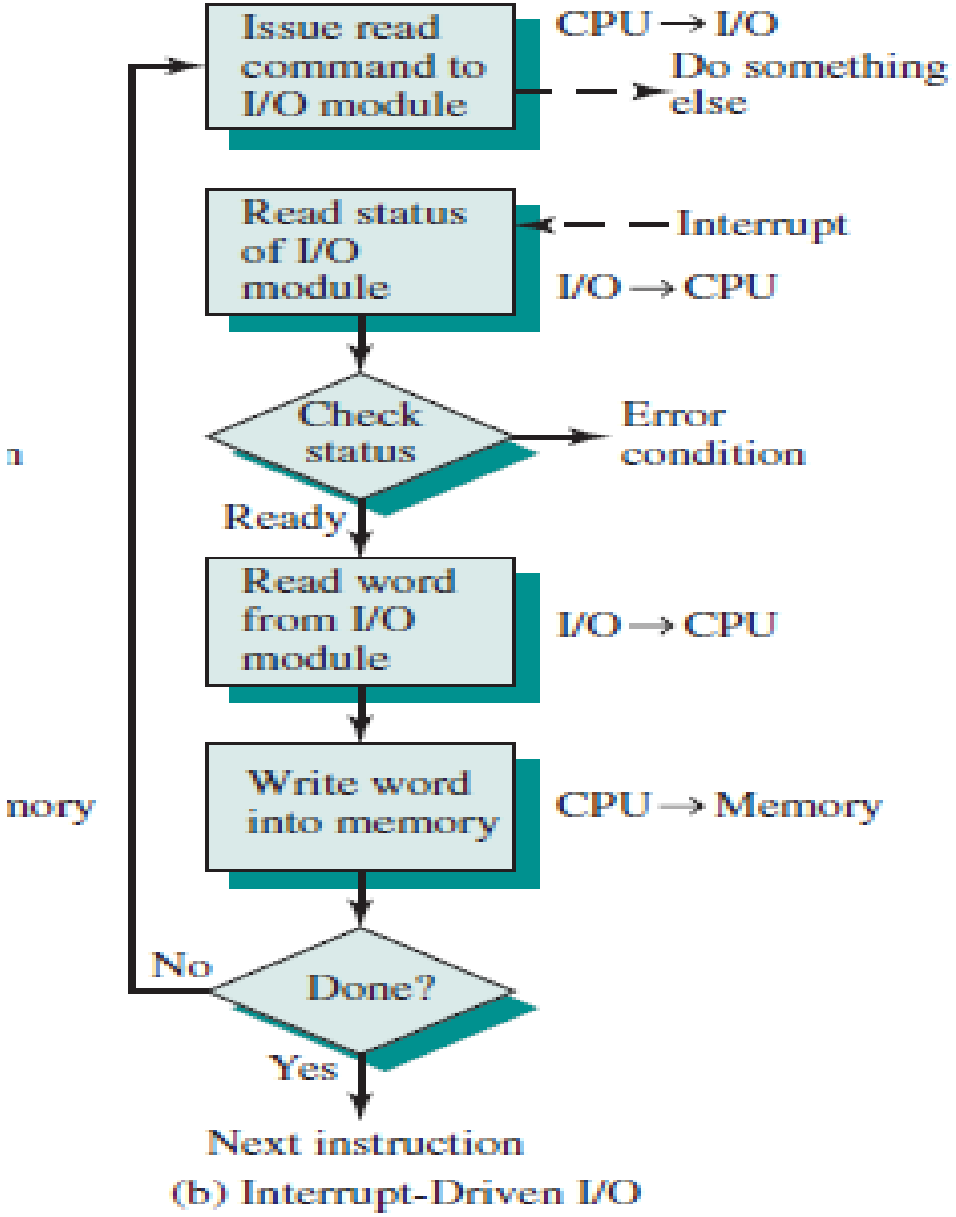
2. isolated

- Memory and I/O have separate address spaces.
- The command line specifies whether the address refers to a memory location or an I/O device.
- The full range of addresses may be available for both.
- Again, with 10 address lines, the system may now support both 1024 memory locations and 1024 I/O addresses.
- Because the address space for I/O is isolated from that for memory,

Interrupt-Drive I/O overview

- For input, The module receives a READ command from the processor.
- It reads data from the peripheral device.
- When data is ready in its data register, It sends an interrupt signal.
- It waits until the processor requests the data.
- It places the data on the data bus.
- It becomes ready for the next I/O operation.

- From the processor’s point of view, the action for input is as follows.
- The processor issues a READ command.
- It then goes off and does something else (e.g., the processor may be working on several different programs at the same time).
- At the end of each instruction cycle, the processor checks for interrupts .
- When the interrupt from the I/O module occurs, the processor saves the context (e.g., program counter and processor registers) of the current program and processes the interrupt.
- In this case, the processor reads the word of data from the I/O module and stores it in memory.
- It then restores the context of the program it was working on (or some other program) and resumes execution.



(b) Interrupt-Driven I/O

Interrupt Processing

- The occurrence of an interrupt triggers a number of events both in the processor hardware and in software.
- When an I/O device completes an I/O operation, the following sequence of hardware events occurs.

1. The device issues an interrupt signal to the processor.

2. The processor finishes execution of the current instruction before responding to the interrupt.

3. The processor tests for an interrupt, determines that there is one, and sends an acknowledgment signal to the device that issued the interrupt. The acknowledgment allows the device to remove its interrupt signal.

4. The processor now needs to prepare to transfer control to the interrupt routine. To begin, it needs to save information needed to resume the current program at the point of interrupt.

(a) the status of the processor, which is contained in a register called the program status **word (PSW**)

(b) the location of the next instruction to be executed, which is contained in the program counter.

These can be **pushed onto the system control stack.**

5. The processor now loads the program counter with the entry location of the

interrupt-handling program that will respond to this interrupt.

- Once the program counter has been loaded, the processor proceeds to the next instruction cycle, which begins with an instruction fetch. control is transferred to the interrupt-handler program.

6. At this point, the program counter and PSW relating to the interrupted program have been saved on the system stack.

7. The interrupt handler next processes the interrupt. This includes an examination of status information relating to the I/O operation or other event that caused an interrupt. It may also involve sending additional commands or acknowledgments to the I/O device.

8. When interrupt processing is complete, the saved register values are retrieved from the stack and restored to the registers .

9. The final act is to restore the PSW and program counter values from the stack. As a result, the next instruction to be executed will be from the previously interrupted program.

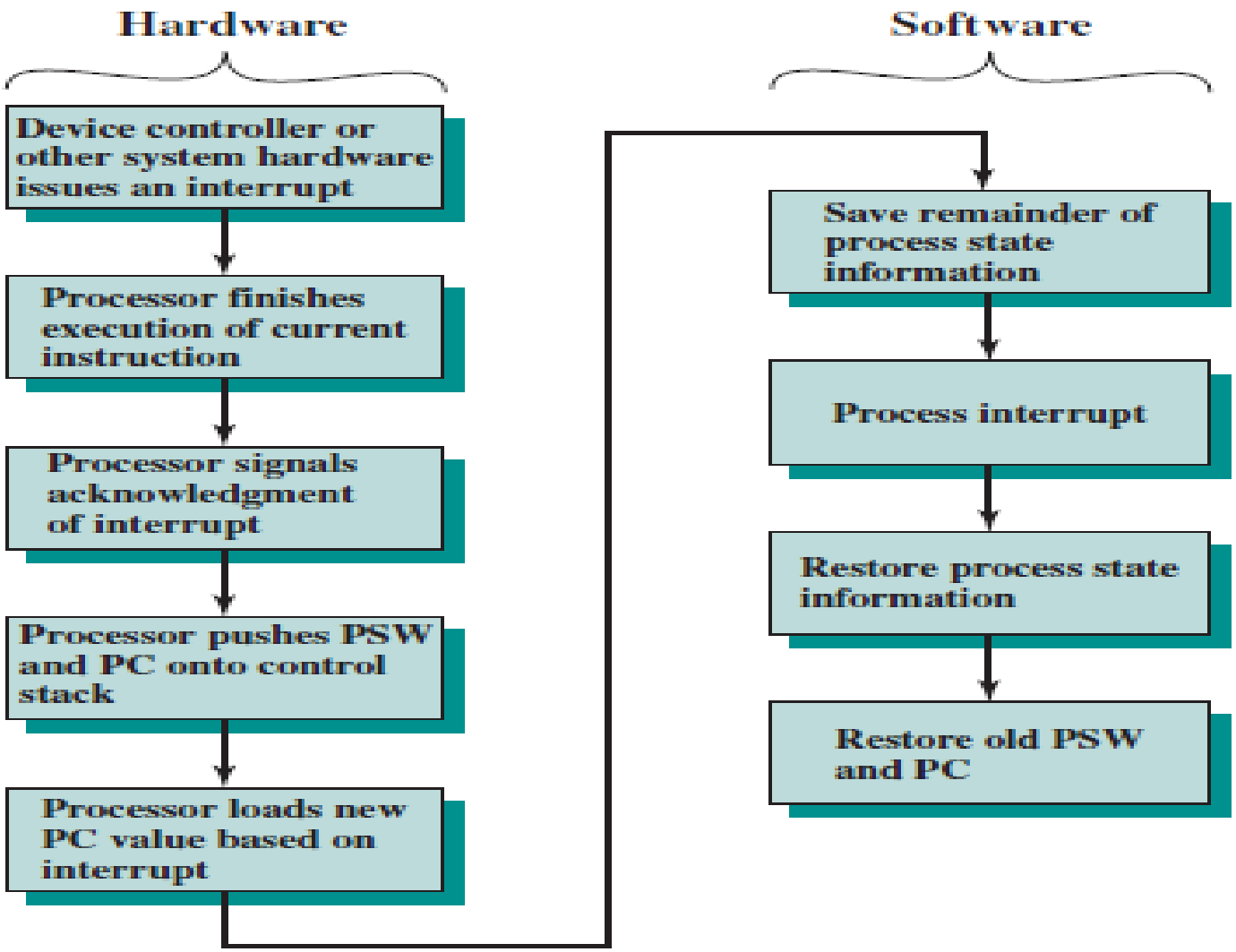


Figure 7.6 Simple Interrupt Processing

Design Issues in Interrupt-Drive I/O

- Two design issues arise in implementing interrupt I/O.

1. There will almost invariably be multiple I/O modules, how does the processor determine which device issued the interrupt?
2. if multiple interrupts have occurred, how does the processor decide which one to process?

- Let us consider device identification first.
- Four general categories of techniques are in common use:

1. Multiple interrupt lines
2. Software poll
3. Daisy chain (hardware poll, vectored)
4. Bus arbitration (vectored)

1. Multiple interrupt lines

- The most straightforward approach to the problem is to provide **multiple interrupt lines** between the processor and the I/O modules.
- However, it is impractical to dedicate more than a few bus lines or processor pins to interrupt lines.
- Consequently, even if multiple lines are used, it is likely that each line will have multiple I/O modules attached to it.
- Thus, one of the other three techniques must be used on each line.

2. Software poll

- When the processor detects an interrupt, it branches to an interrupt-service routine whose job it is to poll each I/O module to determine which module caused the interrupt.
- The poll could be in the form of a separate command line (e.g., TESTI/O).
- In this case, the processor raises TESTI/O and places the address of a particular I/O module on the address lines.
- The I/O module responds positively if it sets the interrupt.
- Alternatively, each I/O module could contain an addressable status register.
- The processor then reads the status register of each I/O module to identify the interrupting module.
- Once the correct module is identified, the processor branches to a device-service routine specific to that device.
- The disadvantage of the software poll is that it is **time consuming**

3. Daisy chain

- For interrupts, all I/O modules share a common interrupt request line.
- The interrupt acknowledge line is daisy chained through the modules.
- When the processor senses an interrupt, it sends out an interrupt acknowledge.
- This signal propagates through a series of I/O modules until it gets to a requesting module.
- The requesting module typically responds by placing a word on the data lines. This word is referred to as a *vector* and is either the address of the I/O module or some other unique identifier.
- In either case, the processor uses the vector as a pointer to the appropriate device-service routine.
- This avoids the need to execute a general interrupt-service routine first. This technique is called a *vectored interrupt.*

4. Bus arbitration (vectored)

- With bus arbitration, an I/O module must first gain control of the bus before it can raise the interrupt request line.
- Thus, only one module can raise the line at a time.
- When the processor detects the interrupt, it responds on the interrupt acknowledge line.
- The requesting module then places its vector on the data lines.

Handling Multiple Interrupts(Priority)

- These techniques also help decide priority:
- **Multiple lines:** Highest-priority line is selected first.
- **Software poll:** The order of polling determines priority.
- **Daisy chain:** The order of modules in the chain determines priority

Drawbacks of Programmed and Interrupt-Driven I/O

1. The I/O transfer rate is limited by the speed with which the processor can test and service a device.

2. The processor is tied up in managing an I/O transfer; a number of instructions must be executed for each I/O transfer.

- When large volumes of data are to be moved, a more efficient technique is required: direct memory access (DMA).

DMA Function

- DMA involves an additional module on the system bus.
- The DMA module is capable of mimicking the processor and, indeed, of taking over control of the system from the processor.
- It needs to do this to transfer data to and from memory over the system bus.
- For this purpose, the DMA module must use the bus only when the processor does not need it *(cycle stealing)*, or it must force the processor to suspend operation temporarily.
- DMA module in effect steals a bus cycle.

DMA BLOCK DIAGRAM

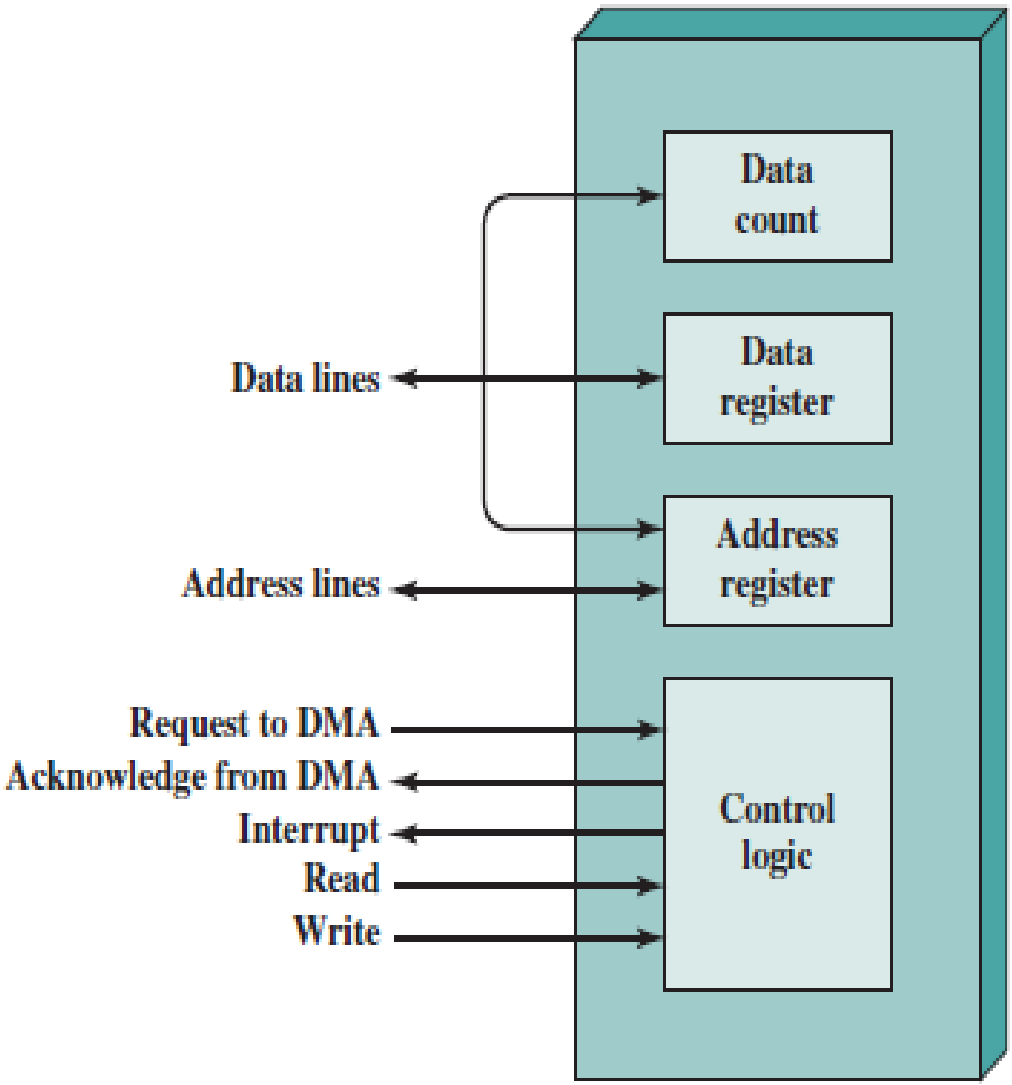


Figure 7.11 Typical DMA Block Diagram

DMA Function

- When the processor wishes to read or write a block of data, it issues a command to the DMA module, by sending to the DMA module the following information:
- Whether a read or write is requested, using the read or write control line between the processor and the DMA module
- The address of the I/O device involved, communicated on the data lines.
- The starting location in memory to read from or write to, communicated on the data lines and stored by the DMA module in its address register
- The number of words to be read or written, again communicated via the data lines and stored in the data count register.

DMA Function

- The processor then continues with other work.
- It has delegated this I/O operation to the DMA module.
- The DMA module transfers the entire block of data, one word at a time, directly to or from memory, without going through the processor.
- When the transfer is complete, the DMA module sends an interrupt signal to the processor.
- Thus, the processor is involved only at the beginning and end of the transfer

DMA Function

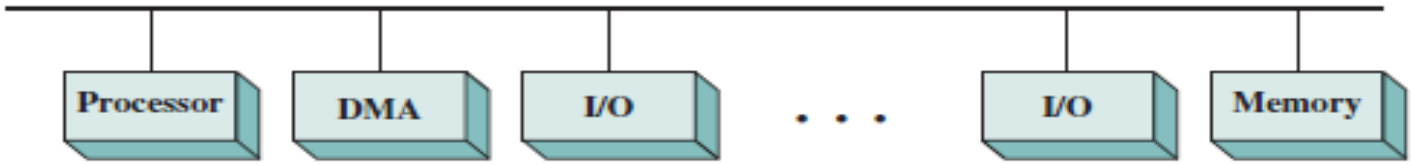
- The processor then continues with other work.
- It has delegated this I/O operation to the DMA module.
- The DMA module transfers the entire block of data, one word at a time, directly to or from memory, without going through the processor.
- When the transfer is complete, the DMA module sends an interrupt signal to the processor.
- Thus, the processor is involved only at the beginning and end of the transfer

DMA and Processor Suspension

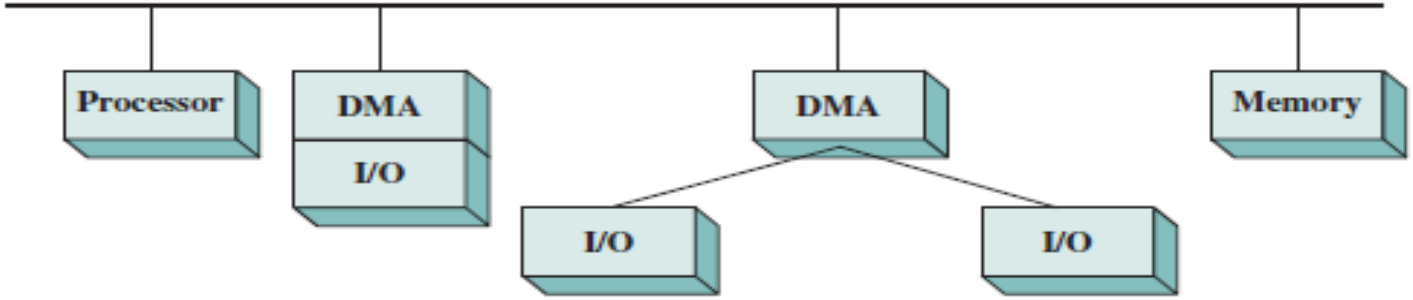
- In the instruction cycle the processor may be suspended.
- In each case, the processor is suspended just before it needs to use the bus.
- The DMA module then transfers one word and returns control to the processor.
- Note that this is not an interrupt; the processor does not save a context and do
- something else.
- Rather, the processor pauses for one bus cycle.
- The overall effect is to cause the processor to execute more slowly.
- for a multiple-word I/O transfer, DMA is far more efficient than interrupt-driven or programmed I/O.

Direct Memory Access-Configuration

- The **DMA mechanism** can be organized In different ways depending on system design.
- These configurations mainly differ in how the DMA module connects with memory and I/O devices.



(a) Single-bus, detached DMA



(b) Single-bus, integrated DMA-I/O

Shared System Bus Configuration

- In this setup:
- All modules (Processor, Memory,I/O,DMA)share the same system bus.
- The DMA module acts like a surrogate processor.
- It uses programmed I/O to transfer data between memory and the I/O module.
- This configuration, while it may be inexpensive, is clearly inefficient.
- As with processor controlled programmed I/O, each transfer of a word consumes two bus cycles.(limited performance)

Single-bus- integrated DMA-I/O

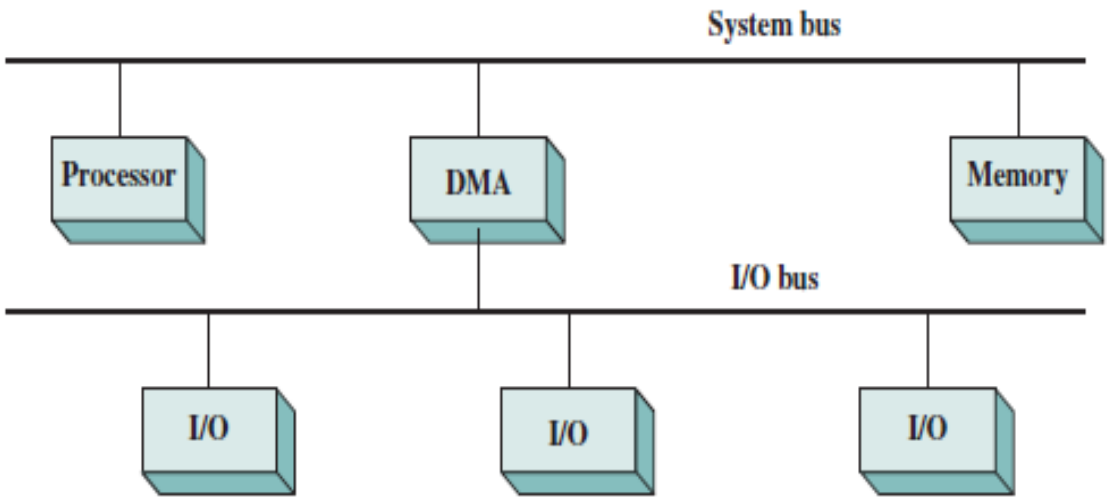
- The number of required bus cycles can be cut substantially by integrating the DMA and I/O functions.
- There is a path between the DMA module and one or more I/O modules that does not include the system bus.

➢ **Alternative DMA Configurations**

- The DMA logic may actually be a part of an I/O module, or it may be a separate module that controls one or more I/O modules.
- This concept can be taken one step further by connecting I/O modules to the DMA module using an I/O bus.

Alternative DMA Configurations

- This reduces the number of I/O interfaces in the DMA module to one and provides for an easily expandable configuration.
- In both of these cases , the system bus that the DMA module shares with the processor and memory is used by the DMA module only to exchange data with memory.
- DMA–I/O communication occurs outside the system bus. This design improves scalability and reduces system bus traffic.



(c) I/O bus